

## A Localization Failures

Failures in this initial stage arise when the tool misinterprets the issue or fails to pinpoint the correct code regions requiring modification. Such errors are often catastrophic, as they misdirect the subsequent repair and validation steps, leaving the tool with little chance of success.

### A.1 Issue Misleading

This failure arises when the tool follows misleading implementation suggestions in the issue description rather than independently diagnosing the underlying problem, leading it to modify incorrect locations in the codebase.

**Instance ID:** scikit-learn\_\_scikit-learn-25102

**Misleading Snippet (from issue description):**

*“Maybe one could adjust the `set_output` to also allow to preserve the dtypes. This would mean one changes the `_SetOutputMixin` to add another argument dtypes to `_wrap_in_pandas_container` ...”*

**Agent Thought Log (misled reasoning).**

THOUGHT: Based on the code and PR description, we need to add a `preserve_dtypes` parameter to `_SetOutputMixin` and modify `_wrap_in_pandas_container` to use the original dtypes when creating the DataFrame. Let's make these changes.

This excerpt from the log shows how the agent, influenced by the issue text, prematurely concluded that the fix should be in the wrapping layer (`_set_output.py`).

**Wrong Focus (direct diff from error log):**

```
1 Agent attempted patch in sklearn/utils/_set_output.py
2 - return pd.DataFrame(data_to_wrap, index=index, columns=columns)
3 + df = pd.DataFrame(data_to_wrap, index=index, columns=columns)
4 + if dtypes is not None:
5 +     # Only set dtypes for common columns
6 +     df = df.astype({col: dtypes[col] for col in common_cols})
7 + return df
```

**Explanation.** As the error log itself and generated diff stated: “The agent was misled by the issue description’s suggested solution, focusing on `_set_output.py` instead of the root cause in `sklearn/base.py`.”

In reality, dtypes had already been discarded earlier inside `_validate_data`, which coerced DataFrames into numpy arrays, so the `base.py` file needs to be modified.

**Correct Fix (essential diff from successful patch) is:**

```
1 - a/sklearn/base.py
2 +++ a/sklearn/base.py
3 @@
4     .....
5 - X = check_array(X, input_name="X", **check_params)
6 + if cast_to_ndarray:
7 + X = check_array(X, input_name="X", **check_params)
8     .....
9 - a/sklearn/base.py
10 +++ a/sklearn/base.py
11 @@
12     .....
13 - return X[:, safe_mask(X, mask)]
```

```

14 + return _safe_indexing(X, mask, axis=1)
15     .....

```

## A.2 Superficial Information Matching

This failure arises when the tool lacks a deep semantic understanding of the codebase and instead depends on shallow heuristics for localization. It can be further divided into the following sub-categories:

**A.2.1 Keywords in Description.** The tool may rely on naive keyword searches derived from issue descriptions, which can misdirect it to functionally related but incorrect files.

**Instance ID:** pylint-dev\_\_pylint-6386

**Misleading Snippet (from issue description):**

*"The short option of the verbose option expects an argument. Also, the help message for the **verbose** option suggests a value **VERBOSE** should be provided."*

**Agent Thought Log (misled reasoning).**

THOUGHT: I see the issue. The verbose option is defined with `_DoNothingAction` but it's not defined as a `store_true` action. Let's modify this to make it a `store_true` action.

This excerpt from the log shows how the agent, influenced by the issue text, prematurely concluded that the fix should only be in `base_options.py`.

**Wrong Focus (direct diff from error log).**

```

1 Agent attempted patch in pylint/lint/base_options.py
2 - "verbose",
3 - {
4 - "action": _DoNothingAction,
5 - .....
6 - },
7 + "verbose",
8 + {
9 + "action": "store_true",
10 .....

```

**Explanation.** As the error log itself showed, patching only `base_options.py` was insufficient. Argument parsing involves multiple layers:

- 1. `argument.py` needed to define a metavar field.
- 2. `arguments_manager.py` and `utils.py` had to propagate that field correctly.
- 3. Both long and short flags (`-verbose`, `-v`) had to be properly registered in `_set_verbose_mode`.

**Correct Fix (essential diff from successful patch).**

```

1 - a/pylint/config/utils.py
2 +++ b/pylint/config/utils.py
3 @@
4 - "--verbose": (False, _set_verbose_mode),
5 + "--verbose": (False, _set_verbose_mode),
6 + "-v": (False, _set_verbose_mode),
7 ...
8 - if not argument.startswith("--"):
9 + if not argument.startswith("-")
10 - a/pylint/config/argument.py

```

```

11  +++ b/pylint/config/argument.py
12  @@
13  -         section: str | None,
14  +         section: str | None,
15  +         metavar: str,
16  ...
17  -         self.kwargs = kwargs
18  +         self.kwargs = kwargs
19  +         self.metavar = metavar

```

**A.2.2 Referred Code.** Issue descriptions often reference specific code snippets as examples, and tools may use the provided example code to identify and modify the corresponding module, even if the true fault lies in a different, interacting module.

**Instance ID:** django\_\_django-14034

**Misleading Snippet (from issue description):**

*“When the form is passed empty values for both sub fields, form.is\_valid() == True. But I expected is\_valid() returns False, because one of the sub fields is set as required.”*

*“A field and a form definition:”*

```

1      class MF(MultiValueField):
2          widget = MultiWidget
3          def __init__(self):
4              fields = [
5                  CharField(required=False),
6                  CharField(required=True),
7              ]
8              widget = self.widget(widgets=[f.widget for f in fields], attrs={})
9              super(MF, self).__init__(fields=fields, widget=widget,
10                                     require_all_fields=False, required=False)
11          def compress(self, value):
12              return []
13
14      class F(Form):
15          mf = MF()

```

**Agent Thought Log (misled reasoning).**

“Based on the repository structure, I can see that the forms module is located in ‘/django/forms’. Let’s first check the current implementation of **MultiValueField**.”

“Let me search for **MultiValueField** in this file.”

“Looking at the code in **MultiValueField.clean()**, the issue is in this part:”

**Wrong Focus (direct diff from error log).**

```

1  - a/django/forms/fields.py
2  +++ b/django/forms/fields.py
3  class MultiValueField(Field):
4      value = self.widget.decompress(value)
5      if not value or isinstance(value, (list, tuple)):
6          if not value or not [v for v in value if v not in self.empty_values]:
7  -         if self.required:

```

```

8 +             # Check if any field is required, even if the MultiValueField
            itself is not
9 +             if self.required or (not self.require_all_fields and any(f.
            required for f in self.fields)):
10                  raise ValidationError(self.error_messages['required'], code='
            required')
11             else:
12                  return self.compress([])

```

**Explanation.** The “runnable reference code” in the issue, along with the problem statement focusing on `form.is_valid()` and `MultiValueField`’s server-side validation path, guides the agent to narrow down the root cause to `MultiValueField.clean()`. The argument is that when all sub-fields are empty, only `MultiValueField.required` is checked, neglecting the `required` attribute of the sub-fields. However, the user-visible “is this field required” experience is also controlled by the propagation logic in the *rendering layer* (HTML `required` attribute). Even if server-side validation acts as a fallback, the absence of front-end validation is still not desirable. The true fix lies in `BoundField.build_widget_attrs()`: when `require_all_fields=False` and `MultiWidget` is used, the “required” semantic needs to be passed down from the parent field to each sub-widget.

**Correct Fix (essential diff from successful patch).**

```

1 - a/django/forms/boundfield.py
2 +++ b/django/forms/boundfield.py

```

**A.2.3 Error Stack Trace.** Tools may also be misled by stack traces from the issue description or their own reproduction scripts, leading them to attempt localized patches without examining the broader call chain to identify the root cause.

**Instance ID:** `sympy__sympy-17318`

**Error Stack Trace (from error log):**

```

Traceback (most recent call last):
  File "sympy\simplify\sqrtdenest.py", line 132, in sqrtdenest
    z = _sqrtdenest0(expr)
  File "sympy\simplify\sqrtdenest.py", line 242, in _sqrtdenest0
    return expr.func(*[_sqrtdenest0(a) for a in args])
  File "sympy\simplify\sqrtdenest.py", line 242, in _sqrtdenest0
    return expr.func(*[_sqrtdenest0(a) for a in args])
  File "sympy\simplify\sqrtdenest.py", line 235, in _sqrtdenest0
    return _sqrtdenest1(expr)
  File "sympy\simplify\sqrtdenest.py", line 319, in _sqrtdenest1
    val = _sqrt_match(a)
  File "sympy\simplify\sqrtdenest.py", line 159, in _sqrt_match
    r, b, a = split_surds(p)
  File "sympy\simplify\radsimp.py", line 1032, in split_surds
    g, b1, b2 = _split_gcd(*surds)
  File "sympy\simplify\radsimp.py", line 1068, in _split_gcd
    g = a[0]
IndexError: tuple index out of range

```

**Agent Thought Log (stack-driven mislocalization).**

“Now I can see the issue. The problem occurs in `_split_gcd` when `a` is an empty tuple. Let’s modify the `_split_gcd` function to handle empty input.”

**Wrong Focus (direct diff from error log).**

```

1 - a/sympy/simplify/radsimp.py
2 +++ b/sympy/simplify/radsimp.py
3 @@
4  def _split_gcd(*a):
5  @@
6  -     g = a[0]
7  +     if not a:
8  +         return S.One, [], []
9  +     g = a[0]
10 ...

```

**Explanation.** The stack trace indeed ends in `_split_gcd()` with an `IndexError`, but the *root cause* is earlier: `_sqrt_match()` in `sqrtidenest.py` was calling `split_surds(p)` under a too-permissive guard (`all((x**2).is_Rational ...)`), which allowed non-positive (or otherwise invalid) surd squares to slip through. That led to `split_surds` producing an empty `surds` sequence and, ultimately, an empty argument list for `_split_gcd(*surds)`. Patching `_split_gcd` masks the symptom; tightening the call site in `_sqrt_match` prevents the invalid call and correctly returns the original expression unchanged when denesting is not applicable.

**Correct Fix (essential diff from successful patch).**

```

1 - a/sympy/simplify/sqrtidenest.py
2 +++ b/sympy/simplify/sqrtidenest.py
3 @@
4  -         if all((x**2).is_Rational for x in pargs):
5  +         sqargs = [x**2 for x in pargs]
6  +         if all(sq.is_Rational and sq.is_positive for sq in sqargs):
7  +             r, b, a = split_surds(p)
8  +             res = a, b, r
9  +             return list(res)

```

**B Repair Failures**

Once a location is identified, the tools generate a code patch to repair the faultss. Failures in this stage can be further divided into the following sub-categories:

**B.1 Fix Strategy Defects.**

This category of failure arises from flaws in the tool's high-level repair strategy, preventing the tool from effectively resolving the issue.

**B.1.1 Specific Case Overfitting.** The generated patch may be narrowly tailored to the specific scenario presented in the issue description, leading to brittle and incomplete patches.

**Instance ID:** `django__django-13551`

**Misleading Snippet (from issue description):**

*"The fix is to add the user's email address into `PasswordResetTokenGenerator._make_hash_value()`."*

**Agent Thought Log (overfitted reasoning).**

"Let's consider any other edge cases: Case sensitivity, Unicode emails, Whitespace..."

**Let's modify our fix to handle these cases."**

**Wrong Focus (direct diff from agent's log).**

```

1 --- a/django/contrib/auth/tokens.py
2 +++ b/django/contrib/auth/tokens.py
3 @@
4 -         # Include email in hash to invalidate token when email changes
5 -         email = getattr(user, 'email', '') or ''
6 -         return str(user.pk) + user.password + str(login_timestamp) + str(
            timestamp) + email
7 +         # Include email with normalization (overfitted)
8 +         email = (getattr(user, 'email', "") or "").lower().strip()
9 +         return str(user.pk) + user.password + str(login_timestamp) + str(
            timestamp) + email

```

**Explanation.** The issue only required *including the user's email* so that *any* email change invalidates existing tokens. The agent overfitted by adding lowercase/whitespace normalization and bespoke “edge-case” tests (e.g. treating Test@Example.com → test@example.com as *no change*). It also hard-coded the field name as email rather than respecting custom email fields, which needs to use `get_email_field_name()` function.

**Correct Fix (essential diff from successful patch).**

```

1 --- a/django/contrib/auth/tokens.py
2 +++ b/django/contrib/auth/tokens.py
3 @@
4 -     def _make_hash_value(self, user, timestamp):
5 -         """
6 -         Hash the user's primary key and some user state that's sure to change
7 -         after a password reset to produce a token that invalidated when it's
8 -         used:
9 +     def _make_hash_value(self, user, timestamp):
10 +         """
11 +         Hash the user's primary key, email (if available), and some user state
12 +         that's sure to change after a password reset to produce a token that is
13 +         invalidated when it's used:
14 @@
15 -         login_timestamp = '' if user.last_login is None else user.last_login.
            replace(microsecond=0, tzinfo=None)
16 -         return str(user.pk) + user.password + str(login_timestamp) + str(
            timestamp)
17 +         login_timestamp = '' if user.last_login is None else user.last_login.
            replace(microsecond=0, tzinfo=None)
18 +         email_field = user.get_email_field_name()
19 +         email = getattr(user, email_field, '') or ''
20 +         return f'{user.pk}{user.password}{login_timestamp}{timestamp}{email}'
21 ...

```

**B.1.2 Evasive Repair.** This failure arises when the tool addresses a bug's symptom by wrapping problematic code in defensive constructs like broad try-except blocks or null checks, rather than fixing the underlying logical flaw. This prevents crashes but masks root causes.

**Instance ID:** django\_\_django-16938

**Problem Context (from issue description):**

*“Serialization of many to many relation with custom manager using select\_related cause FieldError: Field cannot be both deferred and traversed using select\_related at the same time...”*

Traceback (most recent call last):

```
...
django.core.exceptions.FieldError: Field TestTag.master cannot be
both deferred and traversed using select_related at the same time.
```

### Agent Thought Log (evasive reasoning).

THOUGHT: The traceback shows a **FieldError** when `.only("pk")` is used on a queryset that already has `select_related` from a custom manager. This causes a crash. I can prevent the crash by wrapping the `.only("pk")` call in a **try-except block**. If it fails, I'll just ignore the exception and proceed with the original queryset. This will stop the error.

### Wrong Focus (direct diff from agent's log).

```
1 --- a/django/core/serializers/python.py
2 +++ b/django/core/serializers/python.py
3 @@
4 -     return getattr(obj, field.name).only("pk").iterator()
5 +     # Get the queryset and try to optimize it with only('pk')
6 +     # If that fails due to select_related, fall back to the original queryset
7 +     qs = getattr(obj, field.name)
8 +     try:
9 +         qs = qs.only("pk")
10 +     except:
11 +         pass
12 +     return qs.iterator()
```

**Explanation.** The agent correctly identifies that `'only("pk")'` causes a `'FieldError'` when combined with `'select_related'` from the custom manager. However, instead of resolving the underlying query conflict, it opts for an evasive `'try-except pass'` block. This approach merely suppresses the crash. While the serialization no longer fails, it silently discards the `'only("pk")'` optimization whenever `'select_related'` is present, potentially fetching unnecessary data from related models and causing performance degradation. It masks the symptom (the crash) without addressing the root cause (the incompatible query operations).

### Correct Fix (essential diff from successful patch).

```
1 --- a/django/core/serializers/python.py
2 +++ b/django/core/serializers/python.py
3 @@
4 -     return getattr(obj, field.name).only("pk").iterator()
5 +     return getattr(obj, field.name).select_related().only("pk").iterator()
6
7 --- a/django/core/serializers/xml_serializer.py
8 +++ b/django/core/serializers/xml_serializer.py
9 @@
10 -     return getattr(obj, field.name).only("pk").iterator()
11 +     return getattr(obj, field.name).select_related().only("pk").iterator()
```

**B.1.3 Redundant Erroneous Implementation.** The tools may introduce overly complex solutions by re-implementing existing logic or ignoring framework extension points, adding redundant code and potentially new bugs.

**Instance ID:** `django__django-13449`

**Problem Context (from issue description):**

“Lag() with DecimalField crashes on SQLite.”

OperationalError: near "OVER": syntax error

The generated SQL query is:

```
SELECT ..., CAST(LAG("app_lagtest"."amount", 7) AS NUMERIC)
OVER (PARTITION BY ... ) AS "w" FROM "app_lagtest"
```

*“I believe this fails as the CAST() statement ends after LAG whereas it should be around the whole statement...”*

### Agent Thought Log (Redundant/Erroneous Reasoning).

THOUGHT: The SQL generated for a DecimalField inside a window function is incorrect for SQLite. The CAST(... AS NUMERIC) is wrapped only around the LAG(...) part, not the entire ... OVER ... clause. I need to create a special `as_sqlite` method for the Window class. Inside it, **I will manually rebuild the entire SQL string for the window expression.** I’ll deconstruct the expression, compile its parts, and then wrap the reconstructed string in a CAST. To prevent the inner LAG function from also adding a CAST, **I’ll try to temporarily remove its `as_sqlite` method at runtime.**

### Wrong Focus (direct diff from agent’s log).

```
1 --- a/django/db/models/expressions.py
2 +++ b/django/db/models/expressions.py
3 @@
4 +
5 +     def as_sqlite(self, compiler, connection, **extra_context):
6 +         # Agent attempts to manually rebuild the SQL string
7 +         if hasattr(self.source_expression, 'output_field') and self.
8 +             source_expression.output_field.get_internal_type() == 'DecimalField':
9 +             expr_sql, params = compiler.compile(self.source_expression)
10 +            # Brittle attempt to strip a potential inner CAST
11 +            if expr_sql.startswith('CAST('):
12 +                expr_sql = expr_sql[5:expr_sql.find(' AS NUMERIC')]
13 +                window_sql = []
14 +                \# ... redundant manual recompilation of window clause parts ...
15 +                sql = 'CAST(%s OVER (%s) AS NUMERIC)' % (expr_sql, ''.join(window_sql
16 +                    ).strip())
17 +            return sql, params
18 +        \# Recursive call risks infinite loops
19 +        return self.as_sql(compiler, connection, **extra_context)
```

**Explanation.** The agent’s solution is a classic example of redundant and erroneous implementation. It correctly identifies the need for a backend-specific method (`as_sqlite`) but fails to use the existing framework tools. Instead of leveraging Django’s `SQLiteNumericMixin`—a built-in extension point designed specifically for this ‘CAST’ issue—the agent attempts to re-implement the entire SQL generation logic for window functions from scratch. This approach is not only verbose and redundant but also highly fragile, relying on string manipulation (`expr_sql.startswith('CAST(')`) and creating complex logic that was prone to errors, as seen in the logs where it tried and failed to use `delattr` on class methods. It completely missed the simple, idiomatic solution provided by the framework.

## B.2 Implementation Details Defects.

This occurs when the tool’s overall repair strategy is sound, but the implementation contains technical errors, leading to incorrect or failing patches



**B.2.1 Logic Errors.** This group covers fundamental flaws in the patch’s reasoning and execution flow. We identify three distinct sub-categories of such errors: **algorithmic misunderstandings**, flawed **control flow**, and inadequate **boundary handling**.

**B 2.1.1 Algorithmic Implementation.** refers to failures in the concrete design and execution of an algorithm, such as omitting essential steps, misordering operations, or reversing logic, which causes the fix to break intended functionality or introduce new errors.

**Instance ID:** matplotlib\_\_matplotlib-20237

**Problem Context (from issue description):**

*“The spacing for a datetime axis when using usetex=True is unclear in matplotlib version 3.4 when comparing it to 3.3.”* The issue description includes images clearly showing that in version 3.4, the tick labels (e.g., “00:02:30”) are rendered without proper spacing between components, appearing as “00:02:30”.

**Agent Thought Log (Flawed Reasoning).**

THOUGHT: The current `_wrap_in_tex` function uses a complex regex to split the string and wrap parts in `mathdefault`. This seems to be breaking the spacing. **I will replace this complex logic entirely.** A simpler method is to wrap the *entire* string in a single `textrm...` command. This should treat the whole thing as plain text and fix the spacing issues. I also need to escape any special TeX characters to be safe.

**Wrong Focus (direct diff from agent’s log).**

```

1  --- a/lib/matplotlib/dates.py
2  +++ b/lib/matplotlib/dates.py
3  @@
4  -   p = r'([a-zA-Z]+)'
5  -   ret_text = re.sub(p, r'}$\\1$\\mathdefault{', text)
6  -
7  -   # Braces ensure dashes are not spaced like binary operators.
8  -   ret_text = '$\\mathdefault{' + ret_text.replace('-', '{-}') + '$'
9  -   ret_text = ret_text.replace('$\\mathdefault{'}$, '')
10 -   return ret_text
11 +   # Agent's flawed implementation:
12 +   # Escape any special TeX characters
13 +   text = text.replace('\\', r'\\\\').replace('{', r'\{').replace('}', r'\}')
14 +   \# ... more character escaping ...
15 +   # Wrap in to use text mode formatting
16 +   return r'$\textrm{' + text + '}$'

```

**Explanation.** This is a clear algorithmic implementation failure. The agent correctly identifies the function to modify (`_wrap_in_tex`) but misunderstands the original algorithm’s intent. The original logic, which used `mathdefault` and split the string, was designed to ensure date labels used the same font as other math-mode text in the plot. The actual bug was that this algorithm was *incomplete*—it failed to properly handle the spacing for non-alphanumeric characters like spaces, colons, and dashes within the TeX math environment.

Instead of refining the existing algorithm by adding the missing steps, the agent discards it entirely and implements a new, flawed one. Its solution of wrapping the whole string in `{...}` might fix the immediate spacing issue but introduces a new problem: the date labels may now render in a different font from other TeX elements on the plot, defeating the purpose of the original

design. The agent's algorithm is logically incorrect because it solves one symptom by violating a key implicit requirement (font consistency).

**Correct Fix (essential diff from successful patch).**

```

1 --- a/lib/matplotlib/dates.py
2 +++ b/lib/matplotlib/dates.py
3 @@
4 -     # Braces ensure dashes are not spaced like binary operators.
5 -     ret_text = '$\mathdefault{' + ret_text.replace('-', '{-}') + '$'
6 +     # Braces ensure symbols are not spaced like binary operators.
7 +     # Braces ensure symbols are not spaced like binary operators.
8 +     ret_text = ret_text.replace('-', '{-}').replace(':', '{:}')
9 +     # To not concatenate space between numbers.
10 +    ret_text = ret_text.replace(' ', r'\;')
11 +    ret_text = '$\mathdefault{' + ret_text + '$'
```

*B 2.1.2 Control Flow.* occurs when the tool constructs a faulty logical path. This can manifest as incorrect conditional checks, flawed loop logic, or improper branch handling.

**Instance ID:** django\_\_django-14534

**Problem Context (from issue description):**

*"BoundWidget.id\_for\_label() ignores the id available through self.data['attrs']['id']. This re-implementation for rendering the "id" is confusing and presumably not intended... By changing the code [...] to return self.data['attrs']['id'] that function behaves as expected."*

# Original implementation in BoundWidget:

```
def id_for_label(self):
    return 'id_%s_%s' % (self.data['name'], self.data['index'])
```

**Agent Thought Log (Flawed Control Flow).**

THOUGHT: Directly returning self.data['attrs']['id'] might cause a KeyError if the 'id' or even 'attrs' is missing. To make this safe, I will add a check. **If a custom ID exists in the attributes, I will use it. If not, I'll just fall back to the original logic** of generating the ID from the name and index. This way, the code won't crash and will handle both cases.

**Wrong Focus (direct diff from agent's log).**

```

1 --- a/django/forms/boundfield.py
2 +++ b/django/forms/boundfield.py
3 @@ -277,6 +277,13 @@ class BoundWidget:
4
5     @property
6     def id_for_label(self):
7 +         """
8 +         Return the ID to be used for this widget's label.
9 +         If an ID was provided in the widget attrs, use that.
10 +         Otherwise, fall back to the default ID format.
11 +         """
12 +         if 'attrs' in self.data and self.data['attrs'].get('id'):
13 +             return self.data['attrs']['id']
14         return 'id_%s_%s' % (self.data['name'], self.data['index'])
```

**Explanation.** The agent implemented a faulty control flow. While the 'if' branch correctly handles the case where a custom 'id' is provided, the 'else' branch is flawed. Instead of looking up

the correct, dynamically generated ID that should be used as a default, it reverts to the original hardcoded logic, which was the source of the bug in the first place. The agent's logic effectively says, "If the fix is easy, apply it; otherwise, reintroduce the bug." This avoids a crash but fails to solve the root problem for cases without a user-provided ID.

**Correct Fix (essential diff from successful patch).**

```

1 --- a/django/forms/boundfield.py
2 +++ b/django/forms/boundfield.py
3 @@ -277,7 +277,7 @@ def template_name(self):
4
5     @property
6     def id_for_label(self):
7 -         return 'id_%s_%s' % (self.data['name'], self.data['index'])
8 +         return self.data['attrs'].get('id')
9
10    @property
11    def choice_label(self):

```

*B 2.1.3 Boundary Handling.* occurs when the tool's logic does not correctly handle edge cases, boundary conditions, or specific corner business scenarios. The code might work for typical inputs but fails for specific values like empty collections, null inputs, zero, the first/last element in a sequence, or overlooked edge-case business contexts.

**Instance ID:** `django__django-11141`

**Problem Context (from issue description):**

"Allow migrations directories without `init.py` files." The issue states that an old check in Django's migration loader prevents it from recognizing 'migrations' directories that are also "namespace packages" (i.e., they lack an `init.py` file).

```

# The obsolete check in MigrationLoader.load_disk():
if getattr(module, '__file__', None) is None:
    self.unmigrated_apps.add(app_config.label)
    continue

```

**Agent Thought Log (Ignoring Boundaries).**

THOUGHT: The problem description is clear. The check for `getattr(module, '__file__')` is outdated and prevents namespace packages from being used for migrations. The fix is to simply **remove this block of code**. The subsequent logic that adds the app to `self.migrated_apps` should then execute correctly for all migration packages.

**Wrong Focus (direct diff from agent's log).**

```

1 --- a/django/db/migrations/loader.py
2 +++ b/django/db/migrations/loader.py
3 @@ -84,11 +84,6 @@ class MigrationLoader:
4         continue
5         raise
6     else:
7 -         # Empty directories are namespaces.
8 -         # getattr() needed on PY36 and older (replace w/attribute access).
9 -         if getattr(module, '__file__', None) is None:
10 -             self.unmigrated_apps.add(app_config.label)
11 -             continue

```

**Explanation.** This is a classic boundary handling failure. The agent correctly identified and removed the code block that was explicitly causing the error with namespace packages. However, it failed to consider the implications of this removal on the subsequent lines. The original code unconditionally added the app to 'self.migrated\_apps' right after the check. The agent's patch preserved this behavior, which created a new bug at the boundary: an app with an **empty** 'migrations' directory would now be incorrectly classified as "migrated". The logic was not robust enough to distinguish between a package containing migrations and a package that was simply an empty directory.

**Correct Fix (essential diff from successful patch).**

```

1  --- a/django/db/migrations/loader.py
2  +++ b/django/db/migrations/loader.py
3  @@
4
5         migration_names = {
6             name for _, name, is_pkg in pkgutil.iter_modules(module.__path__)
7             if not is_pkg and name[0] not in '_~'
8         }
9
10        self.migrated_apps.add(app_config.label)
11        # New, robust logic handles the empty directory boundary case.
12        if migration_names or self.ignore_no_migrations:
13            self.migrated_apps.add(app_config.label)
14        else:
15            self.unmigrated_apps.add(app_config.label)

```

**B.2.2 Data Processing Errors.** This category refers to mistakes in manipulating or transforming program data. Typical cases include incorrect type casting, variable scope mismanagement (e.g., using a local variable as if it were global), or improper data formatting. Such errors often propagate downstream, resulting in crashes or incorrect behavior.

**Instance ID:** django\_\_django-15741

**Problem Context (from issue description):**

"django.utils.formats.get\_format *should allow lazy parameter*." The issue describes a regression where passing a lazy translated string to the function, such as in a template filter 'some\_date|date:\_('Y-m-d')', causes a crash.

TypeError: getattr(): attribute name must be string in  
django.utils.formats.get\_format.

**Agent Thought Log (Variable Scope Mismanagement).**

THOUGHT: The TypeError happens because getattr receives a lazy object instead of a string. The fix is to convert the format\_type parameter to a string. I see that this parameter is used inside the if use\_l10n: block, so **I'll define a new variable, format\_type\_str, inside that block** and use it for the getattr calls. This should resolve the error.

**Wrong Focus (direct diff from agent's log).**

```

1  --- a/django/utils/formats.py
2  +++ b/django/utils/formats.py
3  @@ -124,19 +124,21 @@ def get_format(format_type, lang=None, use_l10n=None):
4  @@
5
6         val = None
7         if use_l10n:
8             # Variable defined ONLY within this 'if' block's scope.

```

```

8 +         format_type_str = str(format_type)
9         for module in get_format_modules(lang):
10 -             val = getattr(module, format_type, None)
11 +             val = getattr(module, format_type_str, None)
12                 if val is not None:
13                     break
14     if val is None:
15 -         if format_type not in FORMAT_SETTINGS:
16 +         # CRASH: format_type_str is not defined if use_l10n=False
17 +         if format_type_str not in FORMAT_SETTINGS:
18             return format_type
19 -         val = getattr(settings, format_type)
20 -     elif format_type in ISO_INPUT_FORMATS:
21 +         val = getattr(settings, format_type_str)
22 +     elif format_type_str in ISO_INPUT_FORMATS:
23         # ...
24 -         for iso_input in ISO_INPUT_FORMATS.get(format_type, ()):
25 +         for iso_input in ISO_INPUT_FORMATS.get(format_type_str, ()):
26         # ...

```

**Explanation.** This is a classic data processing error caused by variable scope mismanagement. The agent correctly identified the need for type conversion (from a lazy object to a string) but failed to manage the scope of the new variable correctly. By defining ‘format\_type\_str’ \*inside\* the ‘if use\_l10n:’ block, the variable was not created when the function was called with ‘use\_l10n=False’. Consequently, subsequent code that referenced ‘format\_type\_str’ crashed with an ‘**UnboundLocalError**’: local variable ‘format\_type\_str’ referenced before assignment. This failure to handle the data transformation at the correct scope rendered the fix incomplete and buggy.

**Correct Fix (essential diff from successful patch).**

```

1 diff --git a/django/utils/formats.py b/django/utils/formats.py
2 --- a/django/utils/formats.py
3 +++ b/django/utils/formats.py
4 @@ -113,6 +113,7 @@ def get_format(format_type, lang=None, use_l10n=None):
5         use_l10n = settings.USE_L10N
6         if use_l10n and lang is None:
7             lang = get_language()
8 +         format_type = str(format_type) # format_type may be lazy.
9         cache_key = (format_type, lang)
10         try:
11             return _format_cache[cache_key]

```

**B.2.3 Insufficient Domain Knowledge.** This category occurs when the tool fails due to missing knowledge of external frameworks, protocols, or library-specific conventions that are not evident from the local code context. Correct implementation may depend on undocumented conventions or established practices beyond the codebase.

**Instance ID:** django\_\_django-11239

**Problem Context (from issue description):**

“Add support for postgresql client certificates and key to dbshell.” The issue states that while Django’s database backend supports SSL client certificates via ‘DATABASES’ settings, the ‘dbshell’ management command does not pass these credentials to the ‘psql’ command-line tool.

‘OPTIONS’: {

```

        'sslmode': 'verify-ca',
        'sslrootcert': 'ca.crt',
        'sslcert': 'client_cert_chain.crt',
        'sslkey': 'client_key.key'
    }

```

### Agent Thought Log (Flawed External Interaction).

THOUGHT: I need to modify the dbshell command to pass the SSL options from the Django settings to the psql executable. I've located the code in postgresql/client.py. It builds a list of command-line arguments. I should get the SSL settings and add them to this list. **The psql command probably has a generic way to set parameters, so I'll use the -set argument for sslmode, sslcert, and sslkey.**

### Wrong Focus (direct diff from agent's log).

```

1  --- a/django/db/backends/postgresql/client.py
2  +++ b/django/db/backends/postgresql/client.py
3  @@
4      if port:
5          args += ['-p', str(port)]
6  +
7  +         if sslmode:
8  +             args += ['-set=sslmode=' + sslmode]
9  +
10 +         if sslcert:
11 +             args += ['-set=sslcert=' + sslcert]
12 +
13 +         if sslkey:
14 +             args += ['-set=sslkey=' + sslkey]
15 +
16 +         if sslrootcert:
17 +             args += ['-set=sslrootcert=' + sslrootcert]
18 +
19 +         args += [dbname]

```

**Explanation.** The agent's proposed fix fails due to a lack of domain knowledge about the external 'psql' command-line tool. It attempts to pass SSL connection parameters using 'psql's '-set' argument. However, this argument is used for setting 'psql's internal variables, not for configuring the database connection's SSL parameters. The correct and standard method for 'psql' is to use environment variables such as 'PGSSLMODE', 'PGSSLCERT', and 'PGSSLKEY', which the tool automatically reads to configure its connection. The agent's choice of a plausible-sounding but incorrect mechanism for external tool interaction renders the fix non-functional.

### Correct Fix (essential diff from successful patch).

```

1  --- a/django/db/backends/postgresql/client.py
2  +++ b/django/db/backends/postgresql/client.py
3  @@
4  +
5  +         sslmode = conn_params.get('sslmode', "")
6  +
7  +         sslrootcert = conn_params.get('sslrootcert', "")
8  +
9  +         sslcert = conn_params.get('sslcert', "")
10 +
11 +         sslkey = conn_params.get('sslkey', "")
12 +
13 +
14 +         if user:
15 +             args += ['-U', user]
16 +
17 +
18 +         subprocess_env = os.environ.copy()
19 +
20 +         if passwd:

```

```

14         subprocess_env['PGPASSWORD'] = str(passwd)
15 +     if sslmode: subprocess_env['PGSSLMODE'] = str(sslmode)
16 +     if sslrootcert: subprocess_env['PGSSLROOTCERT'] = str(sslrootcert)
17 +     if sslcert: subprocess_env['PGSSLCERT'] = str(sslcert)
18 +     if sslkey: subprocess_env['PGSSLKEY'] = str(sslkey)

```

### B.3 Incomplete Repair

This failure occurs when the tool addresses only part of the problem, leaving other necessary changes unimplemented.

**B.3.1 Ignoring Explicit Dependencies.** This happens when the fix may overlook structural dependencies in the code. For example, the tool may modify a child class without updating the parent class, or adjust one implementer of an interface while neglecting others. Such oversights violate explicit structural contracts of the codebase and can introduce inconsistencies or new bugs.

**B 3.1.1 Inheritance.** This failure occurs when the tool correctly identifies and fixes a bug in a subclass but fails to recognize that the same logical flaw exists in the parent class. The repair is therefore incomplete, as the tool's reasoning does not account for the class hierarchy and shared behavior.

**Instance ID:** django\_\_django-12039

**Problem Context (from issue description):**

*"Creating an index through: index = Index(fields=['-name']...) Will generate the valid but not so pretty CREATE INDEX statement: CREATE INDEX "idx" ON "schema\_author" ("name"DESC)*

*...it was assumed that the col\_suffixes passed to django.db.backends.ddl\_references.Columns would be empty for ascending order but instead it will contain empty strings and thus causing this bug."*

The issue description focuses on scenarios involving descending order (DESC) and operator classes (opclasses), which are handled by the `IndexColumns` subclass, drawing the agent's attention there.

**Agent Thought Log (narrow focus).**

THOUGHT: The issue is in the `IndexColumns.__str__` method. Let's fix it there. After applying the patch to `IndexColumns` and running tests, everything passed, so the fix seems correct.

The agent's log clearly shows its focus was exclusively on the `IndexColumns` subclass. After its tests passed, it incorrectly concluded the fix was complete and correct, without ever inspecting the parent class `Columns` for the same whitespace-handling logic.

**Wrong Focus (direct diff from agent's log).**

```

1  --- a/django/db/backends/ddl_references.py
2  +++ b/django/db/backends/ddl_references.py
3  @@ -112,12 +112,15 @@ class IndexColumns(Columns):
4      def col_str(column, idx):
5          # Index.__init__() guarantees that self.opclasses is the same
6          # length as self.columns.
7  -         col = '{} {}'.format(self.quote_name(column), self.opclasses[idx])
8  +         parts = [self.quote_name(column)]
9  +         if self.opclasses[idx]:
10 +             parts.append(self.opclasses[idx])

```

```

11         try:
12 -             col = '{} {}'.format(col, self.col_suffixes[idx])
13 +             if self.col_suffixes[idx]:
14 +                 parts.append(self.col_suffixes[idx])
15         except IndexError:
16             pass
17 -         return col
18 +         return ' '.join(parts)
19
20     return ' '.join(col_str(column, idx) for idx, column in enumerate(self.
    columns))

```

**Explanation.** The agent correctly diagnosed an improper whitespace issue when generating SQL for index columns. However, it only implemented the fix in the `IndexColumns` subclass, which handles special cases like operator classes. It failed to recognize that the same fundamental string formatting flaw existed in the parent class, `Columns`. The agent's reproduction script and validation were too narrow, confirming only the scenarios mentioned in the issue description. A proper fix required applying the same whitespace-handling logic to both the parent and child classes to ensure consistent and correct SQL generation in all contexts.

**Correct Fix (essential diff from successful patch).**

```

1 diff --git a/django/db/backends/ddl_references.py b/django/db/backends/
    ddl_references.py
2 --- a/django/db/backends/ddl_references.py
3 +++ b/django/db/backends/ddl_references.py
4 @@ -83,10 +83,14 @@ class Columns(TableColumns):
5
6     def __str__(self):
7         def col_str(column, idx):
8 +             col = self.quote_name(column)
9             try:
10 -                 return self.quote_name(column) + self.col_suffixes[idx]
11 +                 suffix = self.col_suffixes[idx]
12 +                 if suffix:
13 +                     col = '{} {}'.format(col, suffix)
14             except IndexError:
15 -                 return self.quote_name(column)
16 +                 pass
17 +             return col
18
19         return ' '.join(col_str(column, idx) for idx, column in enumerate(self.
    columns))
20
21 @@ -114,7 +118,9 @@ def col_str(column, idx):
22     # length as self.columns.
23     col = '{} {}'.format(self.quote_name(column), self.opclasses[idx])
24     try:
25 -         col = '{} {}'.format(col, self.col_suffixes[idx])
26 +         suffix = self.col_suffixes[idx]
27 +         if suffix:
28 +             col = '{} {}'.format(col, suffix)

```



**B 3.1.2 Interface Contract.** This failure occurs when a tool modifies a shared function or method but only updates one of its callers, neglecting other parts of the codebase that rely on the same interface. This results in an inconsistent API and an incomplete fix, as the new behavior is not available through all intended pathways.

**Instance ID:** scikit-learn\_\_scikit-learn-14912

**Misleading Snippet (from issue description):**

*“SparseCoder doesn’t expose max\_iter for Lasso... I guess there should be a way for the user to specify other parameters of the estimator used in SparseCoder other than the ones provided in the SparseCoder.\_\_init\_\_ right now.”*

The issue description is narrowly focused on the SparseCoder class, leading the agent to believe that changes are only required within that class’s context.

**Agent Thought Log (narrow focus).**

THOUGHT: The issue is that when using lasso\_cd algorithm in SparseCoder, the max\_iter parameter is not exposed... The fix would be to add the max\_iter parameter to the SparseCoder class and pass it through to the sparse\_encode function.

THOUGHT: The changes I’ve made: 1. Added the ‘max\_iter’ parameter to the ‘SparseCodingMixin’... 2. Added the ‘max\_iter’ parameter to the ‘SparseCoder.\_\_init\_\_’... The changes are minimal and focused on the specific issue. The agent’s reasoning centers exclusively on the SparseCoder class. It identifies the direct call chain (SparseCoder -> SparseCodingMixin -> sparse\_encode) and threads the new parameter through it. After confirming this single path works, it concludes the task is complete, without checking for other callers of sparse\_encode.

**Wrong Focus (direct diff from agent’s log).**

```

1 # Agent's changes were confined to the SparseCoder and its mixin
2 --- a/sklearn/decomposition/dict_learning.py
3 +++ b/sklearn/decomposition/dict_learning.py
4 @@ -865,11 +865,13 @@
5     def _set_sparse_coding_params(self, n_components,
6                                 transform_algorithm='omp',
7                                 transform_n_nonzero_coefs=None,
8 -                                transform_alpha=None, split_sign=False,
9 -                                n_jobs=None, positive_code=False):
10 +                                transform_alpha=None, split_sign=False, n_jobs=
11 +                                None,
12 +                                positive_code=False, transform_max_iter=1000):
13     self.n_components = n_components
14     self.transform_algorithm = transform_algorithm
15     self.transform_n_nonzero_coefs = transform_n_nonzero_coefs
16     self.transform_alpha = transform_alpha
17 +    self.transform_max_iter = transform_max_iter
18 ...
19 @@ -992,12 +994,13 @@
20     def __init__(self, dictionary, transform_algorithm='omp',
21                 transform_n_nonzero_coefs=None, transform_alpha=None,
22 -                split_sign=False, n_jobs=None, positive_code=False):
23 +                split_sign=False, n_jobs=None, positive_code=False,
24 +                transform_max_iter=1000):
25     self._set_sparse_coding_params(dictionary.shape[0],
26                                   transform_algorithm,
```

```

27                                     transform_n_nonzero_coefs ,
28                                     transform_alpha, split_sign, n_jobs,
29 -                                     positive_code)
30 +                                     positive_code, transform_max_iter)
31     self.components_ = dictionary
32 ...

```

**Explanation.** The agent correctly identified that the `max_iter` parameter needed to be passed to the `sparse_encode` utility function. However, it treated `sparse_encode` as a private helper for `SparseCoder`, failing to recognize it as a shared interface used by multiple high-level components. The correct solution required propagating the parameter not only through `SparseCoder`, but also through other callers like `dict_learning` and `dict_learning_online`. This, in turn, necessitated adding the parameter to the public APIs of the user-facing classes that depend on them, such as `DictionaryLearning` and `MiniBatchDictionaryLearning`. By only patching one implementer of the interface, the agent delivered an incomplete fix that created an inconsistent API.

#### Correct Fix (essential diff from successful patch).

```

1  --- a/sklearn/decomposition/dict_learning.py
2  +++ b/sklearn/decomposition/dict_learning.py
3  @@
4  -         lasso_lars = LassoLars(alpha=alpha, fit_intercept=False,
5  +         lasso_lars = LassoLars(alpha=alpha, fit_intercept=False,
6  +                                 verbose=verbose, normalize=False,
7  +                                 precompute=gram, fit_path=False,
8  +                                 positive=positive, max_iter=max_iter)
9  @@
10 -         code = sparse_encode(X, dictionary, algorithm=method, alpha=alpha,
11 +         code = sparse_encode(X, dictionary, algorithm=method, alpha=alpha,
12 +                               init=code, n_jobs=n_jobs, positive=positive_code,
13 +                               max_iter=method_max_iter, verbose=verbose)
14  @@
15 -         this_code = sparse_encode(this_X, dictionary.T, algorithm=method,
16 +         this_code = sparse_encode(this_X, dictionary.T, algorithm=method,
17 +                                   alpha=alpha, n_jobs=n_jobs,
18 +                                   check_input=False,
19 +                                   positive=positive_code,
20 +                                   max_iter=method_max_iter, verbose=verbose).T
21  @@
22 -         code = sparse_encode(X, dictionary.T, algorithm=method, alpha=alpha,
23 +         code = sparse_encode(X, dictionary.T, algorithm=method, alpha=alpha,
24 +                               n_jobs=n_jobs, check_input=False,
25 +                               positive=positive_code, max_iter=method_max_iter,
26 +                               verbose=verbose)
27  @@
28 -         X, n_components, self.alpha,
29 +         dict_learning(X, n_components, self.alpha,
30 +                       tol=self.tol, max_iter=self.max_iter,
31 +                       method=self.fit_algorithm,
32 +                       method_max_iter=self.transform_max_iter,
33 +                       n_jobs=self.n_jobs,
34  @@

```

```

35 -         U, (A, B) = dict_learning_online(
36 +         U, (A, B) = dict_learning_online(
37             X, self.n_components, self.alpha,
38             n_iter=self.n_iter, method=self.fit_algorithm,
39             method_max_iter=self.transform_max_iter,
40             n_jobs=self.n_jobs, dict_init=dict_init,
41 @@
42 -         U, (A, B) = dict_learning_online(
43 +         U, (A, B) = dict_learning_online(
44             X, self.n_components, self.alpha,
45             n_iter=self.n_iter, method=self.fit_algorithm,
46             method_max_iter=self.transform_max_iter,
47             n_jobs=self.n_jobs, dict_init=dict_init,

```

**B.3.2 Ignoring Semantic Dependency.** The tool may also fail to account for logical dependencies that are not explicitly enforced by the code structure, resulting in partial fixes that leave related parts of the framework broken or out of sync. This includes cases where multiple components must remain consistent or recurring code patterns require simultaneous updates.

**B 3.2.1 Logic Coordination.** This failure occurs when a single conceptual fix requires coordinated changes across multiple, logically connected but physically separate files. The tool may identify and patch some of the required locations but fails to grasp the full scope of the change, leading to a partial and ultimately incorrect patch.

**Instance ID:** sympy\_\_sympy-20438

**Problem Context (from issue description):**

```
«>> Eq(b, c).simplify()
```

```
Traceback (most recent call last):
```

```
...
```

```
File ".../sympy/core/relational.py", line 308, in _eval_simplify
    elif dif.equals(0):
```

```
AttributeError: 'Complement' object has no attribute 'equals'
```

**Agent Thought Log (fragmented reasoning).**

THOUGHT: The issue is confirmed. We can see that... `b.is_subset(c)` returns `None` (incorrect)... and `Eq(b, c).simplify()` raises an error (incorrect). Let's look at the relevant code in the sets module.

THOUGHT: The issue is that there's no specific handler for `ProductSet` in `issubset.py`... Let's add a handler for `ProductSet` in `issubset.py`.

(After multiple failed attempts to patch `issubset.py`)...

THOUGHT: We still have an issue. Let me check if we need to add an `_eval_is_subset` method to `ProductSet` (in `sets.py`).

THOUGHT: Great! The `Eq(b, c).simplify()` error is fixed. However, we still have an issue with `b.is_subset(c)` returning `False` when it should be `True`.

The log shows a piecemeal approach: the agent patches `issubset.py` for subset checks, then `sets.py` for `ProductSet` handling, but misses `comparison.py`, which connects subset logic to equality tests. Without the third piece, equality remains broken.

**Wrong Focus (direct diff from agent's log).**

```

1 --- a/sympy/sets/sets.py
2 +++ b/sympy/sets/sets.py

```

```

3 @@ -777,6 +777,29 @@
4
5     def flatten(self):
6         ...
7 +         return ProductSet(*_flatten(self.sets))
8 +
9 +     def _eval_is_subset(self, other):
10 +         "Return True if self is a subset of other, ..."
11 +         if isinstance(other, FiniteSet):
12 +             # ... long, incorrect implementation ...
13 +             for elements in itertools.product(*component_elements):
14 +                 if elements not in other:
15 +                     return False
16 +             return True
17 +         elif isinstance(other, ProductSet):
18 +             # ...
19 +             return None

```

**Explanation.** The final fix spanned **three files**: 1) `relational.py` to guard against non-Expr objects, 2) `issubset.py` to correctly handle  $\text{ProductSet} \subseteq \text{FiniteSet}$ , 3) `comparison.py` to define equality for arbitrary sets using the subset relation. The agent implemented (1) and (2) but entirely missed (3), leaving equality semantics broken.

**Correct Fix (essential diff from successful patch).**

```

1 --- a/sympy/core/relational.py
2 +++ b/sympy/core/relational.py
3 @@ -302,6 +302,9 @@
4         r = r.func(*[i.simplify(**kwargs) for i in r.args])
5         if r.is_Relational:
6 +             if not isinstance(r.lhs, Expr) or not isinstance(r.rhs, Expr):
7 +                 return r
8             dif = r.lhs - r.rhs
9         ...
10 --- a/sympy/sets/handlers/comparison.py
11 +++ b/sympy/sets/handlers/comparison.py
12 @@ -56,4 +56,4 @@
13
14     @dispatch(Set, Set) # type:ignore
15     def _eval_is_eq(lhs, rhs): # noqa: F811
16         return None
17 +     return fuzzy_and(b.is_subset(a) for a,b in [(lhs,rhs),(rhs,lhs)])
18 --- a/sympy/sets/handlers/issubset.py
19 +++ b/sympy/sets/handlers/issubset.py
20 @@ -1,6 +1,6 @@
21     ...
22 -from sympy.sets.sets import FiniteSet, Interval, Set, Union
23 +from sympy.sets.sets import FiniteSet, Interval, Set, Union, ProductSet
24     ...
25 @@ -133,3 +133,7 @@
26     @dispatch(Rationals, Range)

```

```

27 def is_subset_sets(a, b): # noqa:F811
28     return False
29 +
30 +@dispatch(ProductSet, FiniteSet)
31 +def is_subset_sets(a_ps, b_fs): # noqa:F811
32 +     return fuzzy_and(b_fs.contains(x) for x in a_ps)

```

**B 3.2.2 Recurring Pattern.** This failure occurs when the tool fixes one instance of a bug but fails to recognize and fix other occurrences of the same code pattern or logical flaw elsewhere in the codebase.

**Instance ID:** sympy\_\_sympy-21930

**Problem Context (from issue description).**

*“There are Latex rendering problems within the “secondquant” module, as it does not correctly interpret double superscripts containing the “dagger” command...”*

In [3]: Commutator(Bd(a)\*\*2, B(a))  
Out[3]: 
$$-\left[b_{\{0\}}, b^{\dagger_{\{0\}}^2}\right]$$

*“So, it doesn’t render correctly, and that’s because the double superscript “ $b^{\dagger_{\{0\}}^2}$ ”. It should be correct by adding curly brackets “ $\{b^{\dagger_{\{0\}}^2}\}$ ”*

The issue description gives a minimal example with the Bd (CreateBoson) operator, steering the tool toward this specific implementation.

**Agent Thought Log (narrow focus).**

THOUGHT: Found it! The issue is in the `_latex` method of the `CreateBoson` class. The current implementation returns “ $b^{\dagger_{\{0\}}}$ ” which leads to the double superscript issue... We need to wrap the entire expression in curly braces to fix this.

*(After applying the patch)...*

THOUGHT: Perfect! The LaTeX output now shows  $\{b^{\dagger_{\{0\}}^2}\}$ ... The fix is minimal and focused on the specific issue mentioned in the PR description.

The reasoning shows the tool correctly fixed the CreateBoson case but never checked whether the same bug existed elsewhere. It stopped after the example passed.

**Wrong Focus (direct diff from agent’s log).**

```

1 # Agent only patched the CreateBoson class
2 --- a/sympy/physics/secondquant.py
3 +++ b/sympy/physics/secondquant.py
4 @@ -478,7 +478,7 @@
5         return "CreateBoson(%s)" % self.state
6
7     def _latex(self, printer):
8 -        return "b^{\dagger_{%s}}" % self.state.name
9 +        return "{b^{\dagger_{%s}}}" % self.state.name

```

**Explanation.** The agent successfully performed a “spot fix” on the ‘CreateBoson’ class but failed to generalize. The underlying issue was a recurring pattern: any LaTeX printing method for an operator with a superscript needed to be wrapped in braces to be robust against subsequent exponentiation. This same pattern existed in the ‘CreateFermion’ and ‘AntiSymmetricTensor’ classes within the same file. By focusing only on the class from the example, the agent provided an incomplete solution that left identical bugs in place for other operators.

**Correct Fix (essential diff from successful patch).**

```

1 diff --git a/sympy/physics/secondquant.py b/sympy/physics/secondquant.py
2 --- a/sympy/physics/secondquant.py
3 +++ b/sympy/physics/secondquant.py
4 @@ -218,7 +218,7 @@ def _sortkey(cls, index):
5         return (12, label, h)
6
7     def _latex(self, printer):
8 -         return "%s^{%s}_{%s}" % (
9 +         return "{%s^{%s}_{%s}}" % (
10             self.symbol,
11             "".join([ i.name for i in self.args[1]]),
12             "".join([ i.name for i in self.args[2]]))
13 @@ -478,7 +478,7 @@ def __repr__(self):
14         return "CreateBoson(%s)" % self.state
15
16     def _latex(self, printer):
17 -         return "b^{\dagger}_{%s}" % self.state.name
18 +         return "{b^{\dagger}_{%s}}" % self.state.name
19
20 B = AnnihilateBoson
21 Bd = CreateBoson
22 @@ -939,7 +939,7 @@ def __repr__(self):
23         return "CreateFermion(%s)" % self.state
24
25     def _latex(self, printer):
26 -         return "a^{\dagger}_{%s}" % self.state.name
27 +         return "{a^{\dagger}_{%s}}" % self.state.name
28
29 Fd = CreateFermion
30 F = AnnihilateFermion

```

**B.3.3 Issue Interference.** This failure occurs when the tool is misled by incorrect or overly prescriptive suggestions from the issue description instead of independently validating the intended behavior, producing a flawed patch.

**Instance ID:** sympy\_\_sympy-15599 **Misleading Snippet (from issue description):**

*"Mod(3\*i, 2) should reduce to Mod(i, 2) ... and will do so with a change something like this:"*

```

1 diff --git a/sympy/core/mod.py b/sympy/core/mod.py
2 ...
3 @@ -123,9 +123,11 @@ def doit(p, q):
4         for arg in p.args:
5             both_l[isinstance(arg, cls)].append(arg)
6
7 -         if mod_l and all(inner.args[1] == q for inner in mod_l):
8 +         was = non_mod_l[:]
9 +         non_mod_l = [cls(x, q) for x in non_mod_l]
10 +         changed = was != non_mod_l
11 +         if changed or mod_l and all(inner.args[1] == q for inner in
12             mod_l):
13                 # finding distributive term
14                 non_mod_l = [cls(x, q) for x in non_mod_l]
15                 mod = []

```

15 ...

The issue description not only states the problem but provides a complete, line-by-line implementation suggestion, strongly guiding the agent toward a specific but flawed solution.

#### Agent Thought Log (misled reasoning).

THOUGHT: Looking at the code and the PR description, we need to modify the Mod class... Looking at the **PR description's diff**, we need to modify the part around line 123-128. The key change is to check if the non-modulo terms can be simplified before applying the modulo operation.

*(The agent then applies the suggested diff verbatim).*

THOUGHT: Great! The fix works as expected. Now 'Mod(3\*i, 2)' correctly simplifies to 'Mod(i, 2)'.

The agent's log explicitly states its reliance on the "PR description's diff." It correctly identifies the target location but adopts the proposed implementation without independent analysis. After the specific example from the issue works, it wrongly concludes the fix is correct and complete.

#### Wrong Focus (direct diff from agent's log).

```

1 # Agent blindly applies the diff suggested in the issue description.
2 --- a/sympy/core/mod.py
3 +++ b/sympy/core/mod.py
4 @@ -123,9 +123,11 @@ def doit(p, q):
5         for arg in p.args:
6             both_l[isinstance(arg, cls)].append(arg)
7
8 -         if mod_l and all(inner.args[1] == q for inner in mod_l):
9 +         was = non_mod_l[:]
10 +         non_mod_l = [cls(x, q) for x in non_mod_l]
11 +         changed = was != non_mod_l
12 +         if changed or mod_l and all(inner.args[1] == q for inner in mod_l):
13             # finding distributive term
14 -             non_mod_l = [cls(x, q) for x in non_mod_l]
15             mod = []
16             non_mod = []
17             for j in non_mod_l:
```

**Explanation.** The agent was misled by the overly prescriptive and incorrect patch provided in the issue description. It implemented this patch without validating its underlying logic. The suggested change was a convoluted and indirect way to handle the specific case of 'Mod(3\*i, 2)'. The correct and more general solution is to directly identify any integer coefficients in the product ('non\_mod\_l') and reduce them modulo 'q', which is a more fundamental application of modular arithmetic rules. By trusting the flawed implementation, the agent produced a patch that was unnecessarily complex and not robust.

#### Correct Fix (essential diff from successful patch).

```

1 --- a/sympy/core/mod.py
2 +++ b/sympy/core/mod.py
3 @@ -139,6 +140,17 @@
4         net = prod_mod1*prod_mod
5         return prod_non_mod*cls(net, q)
6
7 +         if q.is_Integer and q is not S.One:
```

```

8 +         _ = []
9 +         for i in non_mod_l:
10 +             if i.is_Integer and (i % q is not S.One - S.One):
11 +                 _.append(i % q)
12 +             else:
13 +                 _.append(i)
14 +             non_mod_l = _
15 +             p = Mul(* (non_mod_l + mod_l))
16
17         # XXX other possibilities?
18
19         # extract gcd; any further simplification should be done by the user

```

### C Iterative Verification Failures

Failure from this stage, particularly in agentic frameworks, involves the loop of testing a patch, interpreting the feedback an agent receives from both its tool interactions and test executions, and refining the solution.

#### C.1 Reproduction or Verification Failure

This arises when the tool fails to set up a valid testing environment or correctly interpret its results.

*C.1.1 Reproduction/Validation Run Failure.* The tool often struggles to reproduce or validate issues because it mishandles project-specific environments and test setups. The tool’s reproduction attempts may prune away essential components (e.g., database backends, configuration files), causing tests to fail for setup reasons. For example, in frameworks like Django or Sphinx, failures commonly arise from missing or misconfigured dependencies, incorrect invocation of test runners, or misinterpretation of project-specific scripts and settings.

**Instance ID:** `pytest-dev__pytest-7324` **Problem Context (from issue description).**

```

"Pytest crashes the interpreter on debug build for 3.8+."
>>> Expression.compile("False")
python: Python/compile.c:3559: compiler_nameop: Assertion ``...'` failed.
[1] 29440 abort (core dumped) python

```

The report makes two critical points: 1. The crash is an internal CPython assertion (debug-only). 2. It appears only on Python 3.8 *debug builds*. A correct fix therefore requires reproducing under that exact environment.

**Agent Thought Log (environment blindness).**

```

(The agent creates a script to reproduce the crash).
ACTION (bash): cd /testbed && python3 reproducer.py
OBSERVATION: [Command finished with exit code 0]
THOUGHT: Hmm, it seems the error doesn't reproduce in this environment because
we're not using a debug build of Python. But we can still implement the fix.

```

**Flawed Reproduction Script (runs silently on release builds).**

```

1 +from _pytest._code import compile
2 +
3 +def test_compile(source, mode="eval"):
4 +    try:

```



```

5 +         result = compile(source, mode=mode)
6 +         print(f"Success compiling {source!r} in {mode} mode")
7 +         if mode == "eval":
8 +             print(f"Result: {eval(result)}")
9 +     except Exception as e:
10 +         print(f"Error compiling {source!r} in {mode} mode: {e}")
11 +
12 + # Test special keywords in eval mode

```

**Explanation.** The agent acknowledged the mismatch—“we’re not using a debug build”—yet nevertheless proceeded “in the dark.” Because the requisite assertion is compiled out of non-debug interpreters, the script never triggered the crash, yielding a false sense of success. Any patch produced in this environment is unverifiable and may miss subtle invariants checked only in debug mode. A robust workflow would first arrange the correct environment (e.g. `./configure --with-pydebug`), rerun the reproducer to obtain a failing signal, then iterate until the assertion no longer fires.

**C.1.2 Insufficient Verification Capability.** validate the correctness of a fix. For example, in cases involving visual outputs, such as Matplotlib plots or web-based renderings, the agent may confirm only that a function runs without errors or that an image is produced, without verifying whether the output content is actually correct.

**Instance ID:** psf\_\_requests-6028

**Problem Context (from issue description).**

*“When using proxies in python 3.8.12, I get an error 407. Using any other version of python works fine...”*

```

import requests

# You will need a proxy to test with,
# I am using a paid service.
r = requests.get('https://example.org/', proxies=proxies)
print(r.status_code)

```

The issue explicitly states that a working proxy server which requires authentication is necessary to reproduce the bug. This signals a critical environmental dependency that the agent must handle.

**Agent Thought Log (environment blindness).**

THOUGHT: Let’s create a script to reproduce the error. We’ll need to create a test script that uses a proxy.

*(Agent creates a script with a placeholder proxy URL).*

OBSERVATION: ... Max retries exceeded with url: / (Caused by ProxyError('Cannot connect to proxy.', NewConnectionError('<urllib3...>: Failed to establish a new connection: [Errno -2] Name or service not known')))

THOUGHT: The test fails because we’re using a non-existent proxy server. This is expected... However, the important part is that our code changes handle the proxy authentication header correctly.

The agent **acknowledges the connection failure but incorrectly dismisses it as unimportant**. It failed to reproduce the actual ‘407’ status code and instead received a connection error, yet it proceeded as if the bug was understood and the patch was valid.

**Flawed Reproduction Script (from agent’s log).**

```

1 import requests
2

```

```

3 # Using a test proxy that requires authentication
4 proxies = {
5     'http': 'http://user:pass@proxy.example.com:8080',
6     'https': 'http://user:pass@proxy.example.com:8080'
7 }
8
9 try:
10     # This will always fail because proxy.example.com does not exist.
11     r = requests.get('https://example.org/', proxies=proxies)
12     print(f"Status code: {r.status_code}")
13 except Exception as e:
14     print(f"Error: {e}")

```

**Explanation.** The agent’s failure was procedural: it could not create a valid environment to reproduce the bug. The issue explicitly required a functioning, authenticated proxy server to trigger the 407 error. The agent instead used a placeholder URL, `proxy.example.com`, which was guaranteed to fail with a `NewConnectionError`. Because its reproduction script failed for environmental reasons, the agent **never actually observed the bug it was supposed to fix**. Its subsequent code modifications were purely speculative, and its validation step was meaningless, as the test continued to fail with the same connection error, **masking whether the underlying 407 issue was resolved**. A correct approach would involve setting up a mock proxy server to simulate the required 407 response.

*C.1.3 Reproduction Output Misreading.* misinterprets the feedback from test scripts. For example, even when a test clearly fails or the reproduction script produces output that is unchanged from previous runs—or obviously does not meet the expected criteria—the agent may incorrectly conclude that the test passed.

**Instance ID:** `scikit-learn__scikit-learn-13124`

**Problem Context (from issue description).**

*“Regarding the shuffle parameter, the documentation states: ‘Whether to shuffle each stratification of the data before splitting into batches’. However, instead of shuffling samples within each stratum, the order of batches is shuffled. As you can see in the output below, 1 is always paired with 11, 2 with 12, 3 with 13, etc. regardless whether shuffle parameter is True or False.”*

The issue describes a subtle bug: the implementation shuffles the generated folds, but *not* the data within each class before creating the folds, which makes the shuffle parameter ineffective for cross-validation.

**Agent Thought Log (misinterpretation).**

ACTION (bash): `cd /workspace/... && python reproduce_stratified_kfold.py`

(Agent observes the verbose output shown below).

THOUGHT: The output for `shuffle=True` is different from `shuffle=False`. The **order of the folds is also different** when the random seed changes. This indicates that the shuffle parameter is having an effect, and the script exits with code 0. I will conclude that the patch has fixed the issue.

The agent only checked fold *order*. It failed to verify whether the **content of each fold**—the sample pairings—was actually shuffled.

Original Test: [ 0 10]   Original Test: [ 1 11]   ...   Original Test: [ 9 19]  
 Test: [ 2 12]   Test: [ 9 19]   ...   Test: [ 5 15]

**The Misreading:** Only the **order** changed; the set  $\{[0, 10], [1, 11], \dots, [9, 19]\}$  is unchanged, showing no intra-class shuffle.

**Explanation.** The agent relied on a superficial signal—different fold order with no crash—and **missed that sample pairings inside folds remained fixed**. Thus it accepted a patch that left the statistical properties of the cross-validator unchanged.

## C.2 Iteration Anomalies

This failure often arises when the tool’s reasoning within the iterative loop becomes dysfunctional, hindering further progress toward a correct solution.

**C.2.1 Non-Progressive Iteration.** The tool may get trapped in repetitive modification cycles, repeatedly editing the same code fragment or configuration file across multiple interaction rounds without yielding meaningful progress. This often manifests as near-identical patches being regenerated with only trivial or cosmetic differences, such as variable renaming or shifting conditionals, while the underlying defect remains unaddressed.

**Instance ID:** sphinx-doc\_\_sphinx-7757

### Problem Context (from issue description).

*“The default value for positional only argument has vanished. When building the following document, the default value for ‘b’ is not shown.”*

```
.. py:function:: foo(a, b=0, /, c=1)
```

The issue points to a failure in rendering the default value (b=0) for a positional-only argument, a syntax feature introduced in modern Python versions.

### Agent Thought Log (iterative fixation).

THOUGHT: After analyzing the code, I can see that the issue is in the `_parse_arglist` function... Let’s fix it by modifying the code to properly handle default values for positional-only parameters.

*(Agent attempts its first fix by moving the separator logic after the parameter node is added).*

```
ACTION (bash): python3 /repo/test_positional_only.py
```

**OBSERVATION: Parameters: ... <desc\_parameter ...><desc\_sig\_name ...>...**

*(The test output shows the default value for b is still missing).*

THOUGHT: There seems to be an issue with the default values still not showing up... Let’s fix the code again.

*(Agent attempts its second fix, which is nearly identical to the original code, undoing its previous change).*

```
ACTION (bash): python3 /repo/test_positional_only.py
```

**OBSERVATION: (Test output is identical to the previous run, still failing)**

THOUGHT: I see the issue now. The problem is that we’re adding the separators in the wrong place. Let’s fix it one more time...

This cycle of minor adjustments and identical failures repeated for over ten steps. The agent remained convinced the solution was to reorder operations within the `_parse_arglist` function, despite consistent evidence that this approach was ineffective.

### Flawed Iterative Patches (from agent’s log).

**Attempt 1 (Action 5): Add Parameter, Then Add Separator Analysis:** The agent moves separator insertion to after the parameter node in hopes the default attaches first.

```
1 --- a/sphinx/domains/python.py
2 +++ b/sphinx/domains/python.py
3 @@
4      if param.default is not param.empty:
```

```

5         ...
6         node += nodes.inline('', param.default, classes=['default_value'],
7                               support_smartquotes=False)
8
9     params += node
10    last_kind = param.kind
11    + # Add separators after adding the parameter node
12    + if param.kind != param.POSITIONAL_ONLY and last_kind == param.POSITIONAL_ONLY:
13    +     # PEP-570: Separator for Positional Only Parameter: /
14    +     params += addnodes.desc_parameter(" ", addnodes.desc_sig_operator("/", '/'))
15    + if param.kind == param.KEYWORD_ONLY and last_kind in (param.POSITIONAL_OR_KEYWORD,...
16    +     # PEP-3102: Separator for Keyword Only Parameter: *
17    +     params += addnodes.desc_parameter(" ", addnodes.desc_sig_operator(" ", '*'))
18    +
19    + last_kind = param.kind

```

*Attempt 2 (Action 9): Revert to Separator First Analysis:* The agent toggles back to inserting separators before the parameter node (no semantic progress).

```

1    --- a/sphinx/domains/python.py
2    +++ b/sphinx/domains/python.py
3    @@
4         last_kind = None
5         for param in sig.parameters.values():
6         + # Add separators before adding the parameter node
7         + if param.kind != param.POSITIONAL_ONLY and last_kind == param.POSITIONAL_ONLY:
8         +     # PEP-570: Separator for Positional Only Parameter: /
9         +     params += addnodes.desc_parameter(" ", addnodes.desc_sig_operator("/", '/'))
10        + if param.kind == param.KEYWORD_ONLY and last_kind in (param.POSITIONAL_OR_KEYWORD,...
11        +     # PEP-3102: Separator for Keyword Only Parameter: *
12        +     params += addnodes.desc_parameter(" ", addnodes.desc_sig_operator(" ", '*'))
13
14        node = addnodes.desc_parameter()
15        ...
16        params += node
17        last_kind = param.kind

```

*Attempt 3 (Action 13): More Complex (but still flawed) Lookahead Analysis:* The agent adds lookahead to detect the “last positional-only” param, still assuming the bug is separator timing.

```

1    --- a/sphinx/domains/python.py
2    +++ b/sphinx/domains/python.py
3    @@
4
5                               support_smartquotes=False)
6
7        params += node
8
9    - # Add separators before adding the parameter node
10   - ...
11   + # Add separators after the current parameter if needed

```

```

11 +         if param.kind == param.POSITIONAL_ONLY and (
12 +             not any(p.kind == param.POSITIONAL_ONLY for p in list(sig.parameters.
13 +                 values())[list(sig.parameters.values()).index(param) + 1:])
14 +             ):
15 +                 # This is the last positional-only parameter, add the / separator
16 +                 params += addnodes.desc_parameter(" ", addnodes.desc_sig_operator("/", '/'))
17 +                 elif param.kind == param.POSITIONAL_OR_KEYWORD and last_kind == param.POSITIONAL_ONLY:
18 +                     # First non-positional-only parameter after positional-only ones
19 +                     params += addnodes.desc_parameter(" ", addnodes.desc_sig_operator(" ", '/'))
20 +                     elif param.kind == param.KEYWORD_ONLY and last_kind in ...
21 +                     ...
22 +                 last_kind = param.kind

```

**Explanation.** The agent correctly identified the rendering function but mislocated the bug. The default for positional-only parameters was being lost *earlier*, during signature parsing in `sphinx/util/inspect.py`. Because the data source was wrong, no amount of reordering separators in `_parse_arglist` could surface the missing default. Despite repeated identical test failures, the agent did not pivot from its flawed hypothesis and remained stuck editing the same block.

**Correct Fix (essential diff from successful patch).**

```

1 --- a/sphinx/util/inspect.py
2 +++ b/sphinx/util/inspect.py
3 @@
4 -def _parse_arglist(text):
5 +def _parse_arglist(text):
6     """
7     Parse a Python function signature from text into a simplified structure
8     used by the domain renderers.
9     """
10    tokens = _tokenize_signature(text)
11    params = []
12    current = None
13    kind = None
14    seen_posonly = False
15 @@
16 -    # handle separators and simple names
17 +    # handle separators and simple names
18    if tok == '/':
19        seen_posonly = True
20        continue
21    if tok == '*':
22        kind = 'kwonly'
23        continue
24 @@
25 -    # parameter name
26 +    # parameter name
27    if _is_name(tok):
28        # flush previous
29        if current:
30            params.append(current)

```

```

31 -         current = {'name': tok, 'kind': 'posonly' if not seen_posonly and
kind is None else ('poskw' if kind is None else kind)}
32 +         current = \{'name': tok, 'kind': ('posonly' if not seen\_posonly and
kind is None else ('poskw' if kind is None else kind))\}
33 +         current['default'] = None
34         continue
35 @@
36 -         # default value
37 -         if tok == '=' and current:
38 -             current['default'] = _read_default(tokens)
39 +         # default value (retain defaults for positional-only before '/' as well)
40 +         if tok == '=' and current:
41 +             current['default'] = _read_default(tokens)
42 +             continue
43 @@
44 -         if current:
45 -             params.append(current)
46 +         if current:
47 +             params.append(current)
48     return params

```

**C.2.2 Blind Strategy Switching.** After a failed attempt, the tool may abruptly abandon its previous approach and switch to an unrelated strategy, ignoring lessons from prior failures. This results in fragmented attempts, where consecutive patches target different parts of the codebase or propose incompatible changes. The lack of continuity prevents cumulative refinement, causing the tool to oscillate between disconnected fixes instead of converging on a viable solution.

**Instance ID:** django\_\_django-13449

**Problem Context (from issue description).**

Lag() with DecimalField crashes on SQLite.

OperationalError: near "OVER": syntax error

The generated SQL query is:

```
SELECT ..., CAST(LAG("app_lagtest"."amount", 7) AS NUMERIC) OVER (PARTITION BY ... ) AS "w"
FROM "app_lagtest"
```

*I believe this fails as the CAST() statement ends after LAG, whereas it should be around the whole statement...*

**Agent Thought Log (Oscillating Reasoning).**

THOUGHT (Initial Strategy): The issue is that SQLite needs the CAST to be around the entire window function. I'll add a custom `as_sqlite` method to the Window class in `expressions.py` to wrap the whole expression.

*(After this causes a double CAST and subsequent monkey-patching attempts fail...)*

THOUGHT (Blind Switch): Let's try a different approach... let's modify the `sqlite3/operations.py` backend to handle this case.

*(After this low-level string manipulation strategy also fails...)*

THOUGHT (Second Blind Switch): Let's try a different approach - let's go back to modifying the `Window` class. This time, I will **manually rebuild the entire window SQL string from scratch** inside its `as_sqlite` method, stripping out any inner CAST.

**Flawed Iterative Patches (Illustrating Strategy Switching).**

*Attempt 1 (Action 22): Flawed Strategy in expressions.py*

```

1 --- a/django/db/models/expressions.py
2 +++ b/django/db/models/expressions.py
3 @@
4 def as_sqlite(self, compiler, connection, **extra_context):
5     Agent tries to prevent a double CAST by deleting the method from the class
6     if hasattr(type(self.source_expression), 'as_sqlite'):
7     source_class = type(self.source_expression)
8     orig_as_sqlite = source_class.as_sqlite
9

```

**Analysis:** This complex and unsafe approach failed with an `AttributeError`. Instead of diagnosing the root cause (that the source expression `Lag` inherits `SQLiteNumericMixin`), the agent abandoned this file entirely.

*Attempt 2 (Action 26): Abrupt Switch to "String Manipulation" in operations.py*

```

1 --- a/django/db/backends/sqlite3/operations.py
2 +++ b/django/db/backends/sqlite3/operations.py
3 @@
4 def compile_sql(self, compiler, sql, params):
5     # Agent switches to a low-level, brittle string search-and-replace
6     if 'CAST(' in sql and ' OVER ' in sql:
7     cast_start = sql.find('CAST(')
8     cast_end = sql.find(' AS NUMERIC)', cast_start) + len(' AS NUMERIC')
9     over_start = sql.find(' OVER ', cast_end)
10    if over_start != -1:
11    over_end = sql.find(')', over_start) + 1
12    sql = ('CAST(' + sql[cast_start + 5:cast_end - len(' AS NUMERIC')] + ...)@*)return sql, params

```

**Analysis:** This patch is a completely different strategy at a much lower level of abstraction. It ignores the structured expression objects and attempts to fix the problem with fragile string slicing. When this failed, the agent abandoned this file too.

**Explanation.** This case is a prime example of Blind Strategy Switching. The agent's reasoning was fragmented and non-progressive. The agent jumped between two completely different parts of the Django ORM with incompatible solutions. There was no refinement of the initial idea; instead, there was a complete abandonment followed by a switch to a conceptually unrelated approach. Because the agent kept discarding its context and starting over with a new idea, it failed to converge on the simple, correct solution. The final working patch was a return to the first strategy, but implemented in an overly complex way that re-implemented existing logic—a direct result of its chaotic and non-linear problem-solving process.

### C.3 Validation Retreat

#### C.3.1 Verification Abandonment. Instance ID: django\_\_django-11433

**Problem Context:** When a `'ModelForm'` field has a model-level `'default'` and is not included in the submitted form data, its value cannot be programmatically set in the form's `'clean()'` method. The `'construct_instance'` function incorrectly prioritizes the model's default over the value in `'cleaned_data'`, preventing derived fields from being set correctly.

**Agent Thought Log (Abandonment After Repeated Failures):** The agent's log shows a protracted and ultimately futile struggle to create a working test environment. This journey spanned 16 actions, culminating in the agent giving up on verification entirely.

### Log Slice: A Chronicle of Failed Verification Attempts

THOUGHT: I apologize for the confusion with the test script. Let me explain the issue and the fix directly... (This marks the point of abandonment. The agent stops trying to create a working test.)

**ACTION 17 (str\_replace\_editor):** The agent applies its patch to ‘models.py’ without a working test.

THOUGHT: The changes look good... The fix is minimal and maintains backward compatibility... No tool use. Submitting.

**Flawed (and Abandoned) Verification Script:** The agent made numerous iterations of its test script, but could never get it to run. The final version before abandonment looked like this:

```

1  from django import forms
2  from django.conf import settings
3  from django.db import models
4
5  # Configure minimal Django settings (This setup still fails)
6  settings.configure(
7      # ...
8      INSTALLED_APPS=['...', 'testapp'],
9      # ...
10 )
11 import django
12 django.setup()
13
14 class TestModel(models.Model):
15     # ...
16     class Meta:
17         app_label = 'testapp'
18         managed = False
19
20 class TestForm(forms.ModelForm):
21     # ...
22     def clean(self):
23         # ... logic to set cleaned_data['status'] ...
24 # ... Test execution that was never successfully run ...
25
```

Listing 1. The final, non-working script before the agent gave up.

**Explanation:** This is a clear case of Verification Abandonment. The agent correctly located the bug and even proposed a logical fix. However, it became completely entangled in the complexities of setting up a minimal but functional Django environment for testing. After 16 consecutive actions failed to produce a working verification script, the agent gave up. It proceeded to apply its patch and submit the solution without ever successfully running a single test to confirm that the bug was present or that its fix was correct. The final submission was based on pure theoretical reasoning, with no empirical evidence of its correctness.

**C.3.2 Verification Weakening.** Faced with environmental or setup issues, the tool may pivot from a functional verification strategy to a weaker, superficial one. Instead of confirming that the target functionality works end-to-end, it alters the test to check only trivial properties—thereby validating an incorrect patch. This produces a false positive: the agent “proves” success only because it lowered the bar for what counts as a passing test.



**Instance ID:** django\_\_django-11239

**Problem Context (from issue description).**

The dbshell command for PostgreSQL did not pass SSL client certificate parameters from Django's settings to the psql command-line tool. The goal was to make dbshell use these credentials. Critically, the standard way to provide these to psql is via environment variables (e.g., PGSSLCERT), not command-line flags.

**Agent Thought Log (Pivoting to a Weaker Test).**

ACTION 6 (bash): python3 reproduce.py

OBSERVATION:

Traceback (most recent call last):

...

FileNotFoundError: [Errno 2] No such file or directory: 'psql': 'psql'

**THOUGHT:** The error is because psql is not installed, but we can see that the SSL parameters are not being passed... At this point, the agent decides to ignore the functional failure and instead weaken its test.

ACTION 8 (str\_replace\_editor): Agent modifies the test script to mock subprocess.run so that it only prints arguments instead of running psql.

**Wrong Focus (Agent's Incorrect Patch).** The agent assumed psql accepted SSL options as command-line flags, producing a fundamentally invalid patch.

```

1 --- a/django/db/backends/postgresql/client.py
2 +++ b/django/db/backends/postgresql/client.py
3 @@
4 +
5 +     Incorrectly append SSL parameters as command line flags
6 +     if 'sslcert' in options:
7 +         args.append('--sslcert=' + options['sslcert'])
8 +     if 'sslkey' in options:
9 +         args.append('--sslkey=' + options['sslkey'])

```

**"Strong" Test (Functional Verification).** Originally, the correct test would execute the constructed psql command, ensuring it could actually run with the provided SSL credentials:

```

1 # This should invoke the real psql binary.
2 DatabaseClient.runshell_db(conn_params)

```

Listing 2. The intended functional verification.

**Weakened Final Test (Argument List Check).** After the FileNotFoundError, the agent rewrote the test to avoid invoking psql. The new version only inspects the argument list, meaning even invalid flags appear to "pass."

```

1 import subprocess
2 real_run = subprocess.run
3
4 def mock_run(args, **kwargs):
5     # Only prints the command instead of executing it
6     print("Would_execute:", ' '.join(args))
7     return real_run(['true']) # Avoids crash
8
9 subprocess.run = mock_run
10 try:
11     DatabaseClient.runshell_db(conn_params)

```

```

12 finally:
13     subprocess.run = real_run

```

Listing 3. The weakened test (mocking subprocess).

**Explanation.** This is a textbook case of *Verification Weakening*. The agent encountered a real barrier (missing `psql`), but instead of addressing it, it degraded its test from:

- **Strong test:** “Does the command execute correctly with SSL options?”
- **Weakened test:** “Did my patch add certain strings to the argument list?”

The weakened test was incapable of catching the agent’s true error—using unsupported flags instead of environment variables. As a result, the flawed patch appeared correct.

**Correct Fix (Essential Diff).** The real solution was to set environment variables, a behavior the weakened test would never validate.

```

1 --- a/django/db/backends/postgresql/client.py
2 +++ b/django/db/backends/postgresql/client.py
3 @@
4 -     args += [dbname]
5 +     subprocess_env = os.environ.copy()
6 +     if sslcert:
7 +         subprocess_env['PGSSLCERT'] = str(sslcert)
8 +     if sslkey:
9 +         subprocess_env['PGSSLKEY'] = str(sslkey)
10 +     if sslrootcert:
11 +         subprocess_env['PGSSLROOTCERT'] = str(sslrootcert)
12 +     if sslmode:
13 +         subprocess_env['PGSSLMODE'] = str(sslmode)

```

## C.4 Context Amnesia

This failure occurs when the agent loses its original objective or its understanding of the code’s current state during a long interaction

**Instance ID:** `pytest-dev__pytest-6197`

**Problem Context (from issue description).**

A regression in `pytest 5.2.3` caused it to incorrectly collect and import any `__init__.py` file it found, even those in non-test packages. This would crash the test run if any such `__init__.py` file contained code that was not importable in the test environment (e.g., an `assert False` or platform-specific imports).

**Log Slice: The Destructive Edit and Subsequent Confusion**

**ACTION 11:** *Agent correctly identifies the target file, `src/_pytest/python.py`.*

**ACTION 12 (create):** *Agent attempts to **create** the file, forgetting it already exists. OBSERVATION:* `<error>File at /repo/src/_pytest/python.py already exists!</error>`

**ACTIONS 13-14:** *Agent tries again, still failing to recognize the file’s existence.*

**ACTION 15 (bash):** `rm -rf src/_pytest/python.py && touch src/_pytest/python.py`  
*(The moment of amnesia. The agent decides the solution to the “file exists” error is to delete the original source file and replace it with an empty one.)*

**ACTION 16 (str\_replace):** *Agent attempts to “edit” the now-empty file by pasting a huge, but incomplete, block of code into it from its context memory.*

**ACTION 17 (bash):** `python3 -m pytest`

**OBSERVATION:** `NameError: name 'PyCollector' is not defined` *(The direct consequence of the destructive edit: the agent's reconstructed file is missing critical class definitions.)*

**ACTIONS 18-23:** *The agent spends multiple turns trying to fix the NameError and subsequent ImportError by pasting more and more missing code back into the file it had deleted, effectively fighting its own self-inflicted damage.*

**Flawed Action (Destructive File Operation)** The agent's most critical failure was not a coding error, but a process error driven by amnesia. Instead of editing the existing file, it performed a destructive operation that wiped out the necessary context for any subsequent fix.

**Action 15:** *Agent deletes a core source file to solve a file-exists error.*

`rm -rf src/_pytest/python.py touch src/_pytest/python.py`

**Explanation** This is a severe and unambiguous case of Context Amnesia. The agent lost its understanding of the code's state, leading to a cascade of self-inflicted errors.

- (1) **Loss of File State Context:** The agent's initial attempts to 'create' an existing file show it had already lost track of the file system's state. Its disastrous decision in Action 15 to delete the file was a flawed attempt to reconcile its internal state ("I need to create this file") with reality ("The file already exists").
- (2) **Loss of File Content Context:** After deleting the file, the agent's attempt to reconstruct it from memory was incomplete. This led to a series of NameError and ImportError exceptions (Actions 17, 22). The agent was no longer solving the original pytest bug; it was now debugging its own broken, incomplete copy of the source code.
- (3) **Shift in Focus:** The majority of the agent's 31 actions were not spent on the original, relatively simple bug. Instead, they were consumed by a chaotic and inefficient process of environmental setup and, more critically, repairing the damage it had caused to the source code. This demonstrates a complete loss of the original mission.